

State-Based Alerting — How It Works

A plain-language guide to the alerting system, for anyone setting up or reviewing a workflow

The problem this replaces

Before this system existed, an alert workflow that noticed "CVR is down" would send basically the same email every time it ran — whether the problem was brand new, exactly the same as an hour ago, getting worse, or had already been fixed three runs ago. Over time this trains people to ignore the emails, because there's no way to tell "new problem" apart from "same thing again."

This system fixes that by giving every individual problem its own memory. It remembers whether a given product's CVR drop is something it has seen before, whether it's changed, and whether it's still happening — and only sends an email when there's actually something new to say.

The core idea: a "finding"

Every time a workflow flags a problem — say, one specific product's CVR dropping — that gets treated as its own individual **finding**. A finding is tracked separately from every other finding, even other problems flagged in the very same run. Two different products dropping in the same run become two completely separate, independently tracked findings.

Each finding moves through a lifecycle of statuses over time:



Status	What it means
absent	Nothing wrong. Either never seen before, or resolved a while ago.

active	Currently breaching. The system is watching it and has (or is about to) notify someone.
recovering	It just had one "clean" (non-breaching) reading, but needs a couple of clean readings in a row before it's trusted as actually fixed.
resolved	Confirmed fixed. A "problem resolved" notification goes out here.
stale	Nobody has seen fresh data about this finding for a long time (not the same as being fixed — more like "we've lost track of it"). Never sends an email on its own.

What can happen to a finding on any given run

Every time the workflow runs, each finding gets compared against what it looked like last time, and one of these labels gets applied:

- **new** — never seen before, or last seen long enough ago that it counts as a fresh problem
- **recurrence** — was marked "resolved" recently, and has now started breaching again
- **escalation** — was already an active problem, and has now gotten meaningfully worse (either a big jump in severity, or crossing into "critical")
- **resolved** — was active, and has now had enough clean readings in a row to count as fixed
- **none** — nothing meaningfully changed since last time (still breaching the same way, or still clean but not enough clean readings yet to resolve)

Only **new**, **recurrence**, **escalation**, and **resolved** are things worth telling someone about. A **none** result means the system stays quiet — this is the main thing that stops repeat spam.

The settings that control this, explained

Breach enter / Breach exit

These decide what counts as "this product has a problem" and what counts as "this product is fine." They're usually set slightly apart from each other on purpose — for example, entering at a 10% drop but only exiting once it's back above a 5% drop. This gap (sometimes called *hysteresis*) stops a finding from flickering open and closed if the real number is hovering right around the threshold.

Consecutive breaches to open / Consecutive clean reads to resolve

"Consecutive breaches to open" controls how many bad readings in a row are needed before something is treated as a real, confirmed problem (rather than a one-off blip).

"Consecutive clean reads to resolve" is the same idea in reverse — how many clean readings in a row are needed before something is trusted as genuinely fixed, not just a lucky single good reading.

Severity tiers

Lets you define extra thresholds beyond the basic breach condition — most commonly a "critical" tier for a much worse drop. Severity tiers are checked top to bottom, and the first one that matches wins. A finding's severity can also be used to trigger the critical-bypass behaviour described below.

Notify: min interval (cooldown)

Once someone has been told about a problem, this is the minimum amount of time before they'll be reminded about it again (assuming nothing else has changed). This is the main anti-spam control for "still broken, nothing new" situations. It deliberately does *not* apply to a problem getting fixed or coming back — those always notify regardless of timing, since they represent genuinely new information, not a repeat.

Burst cap

Caps how many individual emails can go out from a single run. Anything beyond the cap gets grouped into one combined email instead (the "digest"), sent a short while later. Setting this to 0 means every notification always gets grouped into a combined email, never sent individually.

Quiet hours

A "do not disturb" window (e.g. 10pm–7am) during which non-urgent notifications get held rather than sent immediately, and released once the window ends. Critical-severity problems ignore this window entirely (see below).

Let a critical-severity reading skip cooldown/quiet-hours (critical bypass)

When something crosses into critical severity for the first time, it's treated as urgent enough to notify immediately, overriding both the reminder cooldown and quiet hours. The idea is that a genuinely serious problem should never sit in a queue waiting for a timer to expire.

Flap window / Flap max episodes

If the same finding keeps opening and then resolving over and over in a short window (a flaky signal, or a genuinely unstable situation), this detects that pattern and automatically routes those notifications into the combined digest instead of firing off an individual email every single time it flips.

Stale after

If a finding just stops showing up in the data entirely (not fixed, just... not appearing — maybe the product was discontinued, or the report stopped running), this is how long to wait before quietly marking it "stale" in the background. This never sends an email — it's just internal housekeeping so old findings don't sit "active" forever.

How emails actually get sent

Sending an email happens in two separate steps, not one:

Step 1 — Decide: While the workflow runs, the system decides what *should* happen for each finding (send it individually, group it into a digest, hold it, or suppress it) and records that decision.

Step 2 — Deliver: A separate background process (running continuously, independent of any one workflow run) actually sends the emails a little while later, and only then marks the finding as "the user has been told."

This split matters for one important reason: "the user has been told" only becomes true once an email has genuinely gone out — not the moment the system merely decided to send one. If a send fails, it doesn't get incorrectly counted as delivered.

Where to see and control things

- **Findings page** — a list of every tracked problem, its current status, and severity.
- **A finding's detail page** — full history for that one problem, plus manual controls: Acknowledge, Snooze, Mute, and Resolve.
- **Run details page** — for any single workflow run, shows what happened to each finding on that run and why (sent, held, or suppressed, and the reason).

The manual controls, explained

Action	Effect
Acknowledge	"I've seen this." Stops routine reminders, but still notifies if it gets worse or gets fixed.
Snooze	Pauses all notifications for this finding until a chosen time, then resumes as normal.
Mute	Stops all notifications for this finding completely — including escalation and resolution — until someone unmutes it.
Resolve	Manually marks it fixed right now, without waiting for the usual two clean readings. If the real problem is still happening, it will simply reopen on the next run.

One thing worth knowing: the system's memory of a finding is tied to the workflow itself, not to any one saved version of it. Editing and saving a workflow (changing a threshold, tweaking wording, anything) does not reset the problems it's already tracking — they carry straight through.